IMAGE RECOGNITION

Algorithm Comparison on CIFAR-10

BITS Pilani, Hyderabad Campus

AI / ML Internship

Task Date: 29 June 2026

Models Covered

LeNet · AlexNet · VGG16 · ResNet50 · YOLO · Faster R-CNN · K-Means · U-Net

Evaluation Metrics

Accuracy · Precision · Recall

1. INTRODUCTION

This report covers the implementation and evaluation of eight image recognition algorithms on the CIFAR-10 benchmark dataset. The work was carried out as part of the AI/ML internship at BITS Pilani, Hyderabad, with the primary objective of understanding — at a code level — how different model families are built, where their architectural choices come from, and how those choices manifest in real accuracy numbers.

Rather than treating these models as black-box tools, each was either constructed from scratch following the original research papers, or adapted from a pre-trained checkpoint with a clearly reasoned justification. The comparison is therefore not just a metric table — it reflects deliberate design decisions and the engineering trade-offs behind them.

Three broad strategies were used across the eight models:

Approach	Models	What Was Done
From Scratch	LeNet, AlexNet, VGG16, U-Net	Architecture defined layer by layer based on original papers. All weights randomly initialised and optimised entirely on CIFAR-10 training data.
Pre-trained + Fine-tune	ResNet50, Faster R-CNN	ImageNet-trained weights loaded. Base convolutional layers kept frozen; a lightweight classification head attached on top and trained on CIFAR-10.
Feature Extraction	YOLO	Pre-trained YOLOv8 backbone used as a fixed visual encoder. Feature vectors extracted per image; a Logistic Regression classifier trained on those vectors.

All models were evaluated on the same test split with identical metrics, making the comparison directly interpretable.

2. DATASET — CIFAR-10

CIFAR-10 was chosen as the common benchmark because it is small enough to train on in a reasonable time, large enough to reveal real generalisation differences between models, and well-documented in the literature — making it possible to contextualise results against published work.

Specifications

- 60,000 colour images total — 50,000 training, 10,000 test
- 10 balanced classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck
- Image size: 32 × 32 pixels, 3 channels (RGB)
- 6,000 images per class — class imbalance is not a factor in this study

Preprocessing

- Pixel values divided by 255.0 — scales all inputs to the [0, 1] range, which stabilises gradient flow during training
- Labels flattened from shape (N, 1) to (N,) for compatibility with scikit-learn
- One-hot encoding via to_categorical() applied for Keras models that use categorical cross-entropy loss

No data augmentation was applied in this study. This was a deliberate choice — augmentation inflates effective dataset size artificially, and the goal here was to compare architectures on equal footing.

3. MODELS — IMPLEMENTATION DETAILS

3.1 LeNet (1998)

LeNet was the first CNN to be deployed at scale, originally used by AT&T for cheque digit recognition in the 1990s. Yann LeCun formalised it in the 1998 paper "Gradient-Based Learning Applied to Document Recognition." It remains the clearest starting point for understanding convolutional feature extraction — every modern CNN is an evolution of what LeNet established.

Architecture (adapted for 32×32 RGB input)

- Conv2D — 6 filters, 5×5 kernel — detects low-level edges and gradients
- AveragePooling2D (2×2) — spatial downsampling: 32 → 16
- Conv2D — 16 filters, 5×5 — combines edges into shapes
- AveragePooling2D (2×2) — downsampling: 16 → 6
- Flatten — 6×6×16 = 576-dimensional vector
- Dense (120) + ReLU → Dense (84) + ReLU → Dense (10) + Softmax

Adaptation Notes

Original LeNet used grayscale 28×28 input and sigmoid activations throughout. For CIFAR-10: input changed to 32×32 RGB; sigmoid replaced with ReLU (faster convergence, avoids saturation). Architecture depth and filter counts kept identical to the original paper.

Training

- Epochs: 10 | Batch size: 64 | Optimiser: Adam | Loss: Categorical Cross-Entropy
- Total trainable parameters: 83,126

Results — Accuracy: 61.05% | Precision: 61.44% | Recall: 61.05%

At 61%, LeNet is doing well considering it has just two convolutional layers. CIFAR-10 contains textures and multi-part objects that require deeper hierarchical abstraction than two convolutions can provide. This result is the expected baseline — not a failure of LeNet, but a reflection of what a 26-year-old architecture handles against a modern benchmark.

3.2 AlexNet (2012)

Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton entered AlexNet in ILSVRC 2012 and dropped the top-5 error rate from 26.2% to 15.3% — a margin that made the deep learning community stop and pay attention. Published as "ImageNet Classification with Deep Convolutional Neural Networks," it introduced three ideas that became standard across all subsequent architectures: ReLU activations, Dropout regularisation, and large-scale GPU training.

Architecture (adapted for 32×32 input)

- Conv2D (64, 3×3) + BatchNorm + ReLU + MaxPool — 32 → 16
- Conv2D (128, 3×3) + BatchNorm + ReLU + MaxPool — 16 → 8
- Conv2D (256, 3×3) + ReLU + MaxPool — 8 → 4
- Flatten → Dense (1024) + ReLU + Dropout (0.5)
- Dense (512) + ReLU + Dropout (0.5) → Dense (10) + Softmax

Adaptation Notes

The original AlexNet used 11×11 filters on 224×224 images. Applying an 11×11 kernel to a 32×32 image would eliminate most spatial information in the first layer alone. Filters reduced to 3×3 throughout, and Dense layers reduced from 4096 to 1024/512. BatchNormalization added — not in the original paper but now standard practice. These changes preserve the spirit of AlexNet while making it appropriate for small inputs.

Training

- Epochs: 10 | Batch size: 64 | Optimiser: Adam
- Total parameters: approximately 5 million (reduced from ~60M in original)

Results — Accuracy: 73.99% | Precision: 75.17% | Recall: 73.99%

A 13-point improvement over LeNet, driven by greater depth, more filters per layer, and Dropout preventing the overfitting that would otherwise dominate with a dense classifier head. The result tracks with expectations — AlexNet-class depth is well matched to CIFAR-10 complexity.

3.3 VGG16 (2014) — Best Performing Model

Karen Simonyan and Andrew Zisserman at Oxford's Visual Geometry Group published "Very Deep Convolutional Networks for Large-Scale Image Recognition" in 2014. VGG16 finished second in ILSVRC 2014 and became arguably more influential than the winner — its clean, uniform design made it the architecture of choice for transfer learning and as a feature extractor backbone for years afterward. The "16" refers to 16 layers with learned weights.

The central idea is deceptively simple: use only 3×3 convolutions, stacked deeply, with no variation. Two stacked 3×3 layers cover the same receptive field as one 5×5 layer but with fewer parameters and one additional non-linearity — which makes the decision boundary richer without adding compute proportionally.

Architecture (adapted — 3 blocks for 32×32 input)

- Block 1: Conv2D(64) + BN + Conv2D(64) + BN + MaxPool → 16×16
- Block 2: Conv2D(128) + BN + Conv2D(128) + BN + MaxPool → 8×8
- Block 3: Conv2D(256) + BN + Conv2D(256) + BN + MaxPool → 4×4
- Flatten → Dense(512) + Dropout(0.5) → Dense(256) + Dropout(0.5) → Dense(10) + Softmax

Why 3 Blocks, Not 5

The original VGG16 uses 5 pooling blocks, designed for 224×224 inputs. Applying 5 MaxPool (2×2) layers to a 32×32 image yields a 1×1 feature map — a single number per channel. That collapses all spatial structure before the

classifier even starts. Three blocks reduce 32 → 4, which preserves meaningful spatial relationships while still achieving deep abstraction.

Training

- Epochs: 10 | Batch size: 64 | Optimiser: Adam
- BatchNormalization applied after every convolutional layer

Results — Accuracy: 78.81% | Precision: 81.72% | Recall: 78.81%

VGG16 is the strongest model in this study. Depth, uniform filter design, and per-layer batch normalisation compound: each block extracts progressively more abstract features, and the consistent 3×3 structure means no single layer is asked to do too much. The progression LeNet 61% → AlexNet 74% → VGG16 79% is not coincidental — it reflects what additional depth and regularisation systematically deliver.

3.4 ResNet50 (2015)

Microsoft Research published "Deep Residual Learning for Image Recognition" in 2015, winning ILSVRC that year with a top-5 error of 3.57%. The innovation was the residual (skip) connection: a shortcut that adds the input of a block directly to its output, allowing gradients to flow unchanged through the shortcut path during backpropagation. This solved the degradation problem that had prevented training networks beyond roughly 30 layers.

Implementation Approach — Transfer Learning

ResNet50 pre-trained on ImageNet was loaded from tf.keras.applications. The convolutional base was frozen entirely. A classification head was attached: GlobalAveragePooling2D → Dense(256) + ReLU + Dropout(0.5) → Dense(10) + Softmax. Only this head was trained on CIFAR-10.

Results — Accuracy: 31.24% | Precision: 29.58% | Recall: 31.24%

Why the Result Is Low — and What It Tells Us

ResNet50 is not a weak model. On ImageNet-resolution images (224×224), it achieves over 75% top-1 accuracy. The issue here is architectural mismatch: ResNet50 contains strided convolutions and max-pooling in its early layers that were designed to progressively halve a 224×224 input. Applied to 32×32 images, feature maps collapse to near 1×1 after just a few blocks — the spatial structure that makes convolutional features meaningful is gone before it reaches the bottleneck layers.

This is an important finding: the quality of a pre-trained model is irrelevant if the input resolution is incompatible with what the architecture expects. ResNet50 loaded with frozen weights on 32×32 CIFAR-10 images is not a fair test of ResNet50 — it is a demonstration of why input preprocessing must align with architectural design.

3.5 YOLO — YOLOv8n (Feature Extraction Mode)

"You Only Look Once" was introduced by Joseph Redmon et al. in 2016. Its defining characteristic is single-pass detection: the image is processed once through the network, which simultaneously predicts bounding boxes and

class probabilities for all objects. YOLOv8n (nano) is the smallest variant in the current Ultralytics series, optimised for speed at the cost of some accuracy.

Implementation Approach

YOLO is not a classification model and cannot be used directly for single-label image prediction. The backbone was used as a fixed feature extractor via model.embed() — a function that returns the internal feature representation without running the detection head. These feature vectors were passed to a Logistic Regression classifier. To manage extraction time, 5,000 training images and 1,000 test images were used.

Results — Accuracy: 54.60% | Precision: 53.93% | Recall: 54.36%

54.6% is a reasonable result given the constraints. YOLO's backbone encodes spatial and semantic features tuned for detection — locating multiple objects across a scene — which produces a different feature distribution than classification-focused backbones. Using 10% of the training data also limits what the Logistic Regression can learn. With full training data and a more appropriate classifier, the result would improve.

3.6 Faster R-CNN

"Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks" (Ren et al., 2016, Microsoft Research) replaced the external selective search step in R-CNN with a Region Proposal Network that shares convolutional features with the detection head. This made the pipeline end-to-end trainable and significantly faster. VGG16 was the backbone used in the original paper.

Implementation Approach

Pre-trained VGG16 (weights=imagenet, include_top=False, pooling=avg) was used as the backbone, consistent with the original Faster R-CNN paper. ImageNet preprocessing was applied to CIFAR-10 images. 512-dimensional feature vectors were extracted for all 60,000 images, normalised with StandardScaler, then classified with Logistic Regression (max_iter=3000).

Results — Accuracy: 66.65% | Precision: 66.32% | Recall: 66.65%

VGG16 as a backbone handles variable input sizes more gracefully than ResNet50, because its architecture does not contain the aggressive early striding that collapses feature maps on small images. The 66.65% result reflects the quality of ImageNet-learned VGG16 features applied to a classification problem — solid, without being optimised for it.

3.7 K-Means Clustering

K-Means is an unsupervised clustering algorithm. It does not use labels during training — it groups data points by minimising within-cluster variance. Its inclusion here is deliberate: it serves as the lower-bound baseline, showing what a completely feature-agnostic method achieves on a visual classification problem.

Implementation Approach

- Images flattened: 32×32×3 = 3,072 values per image
- K-Means fit on all 50,000 training images, K=10 (one cluster per class)
- Each cluster mapped to its majority class label via mode vote

- Test images classified by nearest cluster centroid

Results — Accuracy: 22.18% | Precision: 13.18% | Recall: 22.18%

22% is only slightly above random chance (10%). Raw pixel similarity is not a reliable proxy for semantic class membership — two cats photographed in different environments share almost no pixel-level similarity, while a black cat and a black dog might cluster together because of colour. K-Means has no mechanism to learn what the differences between classes actually look like. This result cleanly motivates why supervised, gradient-based feature learning exists.

3.8 U-Net

U-Net was published by Ronneberger, Fischer, and Brox at the University of Freiburg in 2015 ("U-Net: Convolutional Networks for Biomedical Image Segmentation"). It was designed for scenarios with very limited labelled data — common in medical imaging. Its encoder-decoder structure with skip connections means that fine spatial detail from early encoder layers is directly accessible to the decoder, enabling precise per-pixel predictions.

Architecture (adapted for classification)

- Encoder: Conv2D(32) → MaxPool → Conv2D(64) → MaxPool
- Bottleneck: Conv2D(128)
- Decoder: UpSampling → Concatenate(encoder output) → Conv2D(64)
- UpSampling → Concatenate(encoder output) → Conv2D(32)
- Classification head: Flatten → Dense(256) + Dropout(0.5) → Dense(10) + Softmax

Skip Connections

The concatenation steps in the decoder are the defining feature of U-Net. They provide the decoder with direct access to the spatial feature maps computed by the encoder at the same resolution — something a standard CNN discards through pooling. For classification, this gives the model a richer representation: high-level semantics from the bottleneck combined with local spatial detail from the encoder.

Results — Accuracy: 70.91% | Precision: 71.04% | Recall: 70.91%

70.91% is a strong result for a model adapted from segmentation to classification. The skip connections appear to benefit classification — the model retains spatial context that a pure encoder would lose, and this helps it distinguish between visually similar classes like Cat and Dog, or Automobile and Truck.

4. RESULTS

All models evaluated on the CIFAR-10 test set (10,000 images). Precision and Recall computed as macro-average across all 10 classes.

Model	Accuracy	Precision	Recall	Approach
LeNet	61.05%	61.44%	61.05%	From Scratch
AlexNet	73.99%	75.17%	73.99%	From Scratch
VGG16	**78.81%**	**81.72%**	**78.81%**	**From Scratch**
ResNet50	31.24%	29.58%	31.24%	Pre-trained
YOLO	54.60%	53.93%	54.36%	Feature Extraction
Faster R-CNN	66.65%	66.32%	66.65%	Pre-trained
K-Means	22.18%	13.18%	22.18%	Unsupervised
U-Net	70.91%	71.04%	70.91%	From Scratch

4.1 Analysis

From-Scratch Models — Depth Determines Outcome

The three from-scratch classification CNNs (LeNet, AlexNet, VGG16) show a monotonic improvement as architecture depth increases. LeNet's two convolutional layers saturate at 61% — the model simply lacks the capacity to represent CIFAR-10's variety. AlexNet adds three more layers and introduces Dropout and BatchNorm, gaining 13 points. VGG16 stacks eight convolutional layers in a systematic pattern and gains another 5 points over AlexNet. The trend is clean: on a dataset of this complexity, depth directly translates to representational power, and representational power translates to accuracy.

U-Net — Segmentation Architecture Transfers Well

U-Net at 70.91% sits between AlexNet and VGG16, which is a notable result for a model designed for a different task entirely. The skip connections appear to be the reason — by preserving spatial detail from early encoder layers, U-Net gives its classifier head richer input than a pure downsampling encoder would. This suggests that encoder-decoder architectures are worth considering for classification when the classes are visually similar, not just for segmentation.

ResNet50 — Architectural Mismatch on Small Inputs

ResNet50's 31.24% is the most instructive result in this study. It is not a reflection of ResNet50's quality — it is a direct consequence of applying an architecture designed for 224×224 images to 32×32 inputs without resizing. The early strided convolutions in ResNet50 are calibrated for large inputs; on 32×32, they eliminate spatial structure before the deeper layers have anything useful to work with. The frozen pre-trained weights compound the problem — those weights encode features learned at 224×224 resolution, which do not transfer cleanly to a different spatial scale. This outcome reinforces a fundamental engineering principle: pre-trained weights and input resolution must be matched.

Detection Backbones as Classifiers

Both YOLO and Faster R-CNN were adapted from detection architectures to classification via feature extraction. Faster R-CNN (VGG16 backbone, 66.65%) outperforms YOLO (54.60%) for two reasons: the VGG16 backbone

produces richer classification-oriented features than YOLO's detection-oriented encoder, and Faster R-CNN used the full 50,000 training images while YOLO was limited to 5,000. Neither result is a measure of how well these models detect objects — that is not their task here.

K-Means — The Cost of No Feature Learning

K-Means at 22.18% quantifies the gap between pixel-level similarity and semantic similarity. It is above chance (10%) because some classes have consistent dominant colours or textures that cluster naturally — sky in airplane images, grass in frog images. But within-class pixel variance is too large for clustering to work reliably. Every other model in this study — including the weakest CNN — outperforms K-Means by a wide margin, which is precisely the point.

5. BEST PERFORMING MODEL — VGG16

VGG16 achieved 78.81% accuracy and 81.72% precision — the highest scores in this study. The result is not surprising given the architecture, but it is worth unpacking why.

The 3×3 filter design is the most important factor. It might seem counterintuitive that smaller filters outperform larger ones, but the receptive field argument is clear: two 3×3 layers stacked have a 5×5 effective receptive field with fewer total parameters and one additional ReLU non-linearity between them. That extra non-linearity means the network can model a richer function at the same depth. VGG16 applies this principle consistently across all 8 convolutional layers, and the compound effect is evident in the accuracy.

BatchNormalization after every layer is the second key factor. It keeps activations on a scale where gradients can flow efficiently, which is particularly important when training 8 convolutional layers from random initialisation. AlexNet without BatchNorm reached 74%; VGG16 with it reaches 79% — roughly what the normalization contributes.

The adaptation choice — 3 pooling blocks instead of 5 — also matters. Preserving a 4×4 feature map before the classifier gives the Dense layers meaningful spatial input. This is not a deviation from VGG16's design philosophy; it is the correct application of that philosophy to a smaller input resolution.

6. LIMITATIONS AND SCOPE FOR IMPROVEMENT

Several findings in this study point to specific, actionable improvements — not general engineering upgrades, but targeted changes to address identified shortcomings.

ResNet50 — Input Resolution Alignment

The primary fix for ResNet50's 31.24% is to resize CIFAR-10 images to 224×224 before feeding them into the network. This aligns the input with the spatial scale the pre-trained weights were optimised for and should bring performance up to the 75–85% range that ResNet50 achieves on comparable benchmarks. Alternatively, fine-tuning the full network (rather than keeping the base frozen) at a low learning rate could allow the weights to adapt to the smaller resolution, though resizing is the cleaner solution.

All From-Scratch Models — Limited Training Data Coverage

CIFAR-10 provides 5,000 images per class for training. That is sufficient to learn coarse discriminative features but not enough to cover the full within-class variance — different poses, backgrounds, lighting conditions, and scales. Data augmentation (random horizontal flips, moderate rotation, slight zoom) would expose each model to more representative variation during training without requiring additional labelled data. Based on the literature, augmentation typically yields 4–8 percentage point improvements on CIFAR-10 for architectures in this accuracy range.

YOLO — Classifier and Dataset Size

The YOLO experiment used only 5,000 of 50,000 training images due to extraction time. Using the full training set and replacing Logistic Regression with a small Multi-Layer Perceptron (two Dense layers) would both be meaningful improvements. The underlying YOLO backbone is expressive — the bottleneck is the limited training coverage, not the feature quality.

VGG16 and AlexNet — Extended Training

Both models were trained for 10 epochs, which is sufficient to demonstrate convergence but not to reach peak accuracy. The validation accuracy curves for both models are still trending upward at epoch 10, suggesting that additional epochs with a learning rate scheduler (reducing learning rate when validation plateaus) would yield further improvement. A learning rate scheduler combined with 30–50 epochs could realistically add 3–5 percentage points to both models.

Ensemble — Combining Complementary Models

VGG16, U-Net, and AlexNet make different types of errors — each has learned a somewhat different internal representation. A simple majority-vote ensemble of these three models would reduce the individual error rate wherever at least two of the three agree. Ensembles of this type consistently outperform their individual members by 2–4 percentage points on CIFAR-10, which would push the combined accuracy above 81%.

7. REFERENCES

Research Papers

- LeCun, Y. et al. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278–2324.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems (NeurIPS), 25.
- Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. International Conference on Learning Representations (ICLR).
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. Microsoft Research.
- Ren, S., He, K., Girshick, R., & Sun, J. (2016). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015.

Dataset

- Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto.
- https://www.cs.toronto.edu/~kriz/cifar.html

8. CONCLUSION

Eight models spanning six decades of computer vision research were implemented and evaluated against a common benchmark. The results surface several findings that are more useful than the accuracy numbers alone.

Depth and architectural regularity, not raw complexity, determined from-scratch performance. VGG16's systematic 3×3 filter stacking outperformed AlexNet's larger, more varied architecture — which in turn outperformed LeNet's shallower design. The relationship was monotonic and predictable.

Input resolution is a hard constraint for pre-trained models. ResNet50, which should be the strongest model in this study, produced the weakest result among supervised approaches because 32×32 images are incompatible with its internal spatial reduction schedule. This is not a nuance — it is a blocking issue, and it highlights that transfer learning requires explicit attention to input preprocessing, not just weight loading.

Unsupervised methods cannot substitute for feature learning on vision tasks. K-Means at 22% quantifies the cost of having no gradient signal — the gap between it and even the simplest CNN (LeNet, 61%) is 39 percentage points.

Architectures transfer across tasks. U-Net was designed for medical image segmentation; adapted to classification, it scored 70.91% — competitive with purpose-built classifiers of similar depth. Skip connections are a general structural idea, not one bound to segmentation.

The most impactful next step for this study is resizing CIFAR-10 images to 224×224 for ResNet50 evaluation, and applying data augmentation across the from-scratch models. Both changes are targeted, motivated by specific findings in this work, and likely to produce substantial accuracy gains.